

AI Computing Infra, Scaling Laws

CS6501: AI Systems

Fall 2026

Lecture 6

Yue Cheng



UNIVERSITY
of VIRGINIA

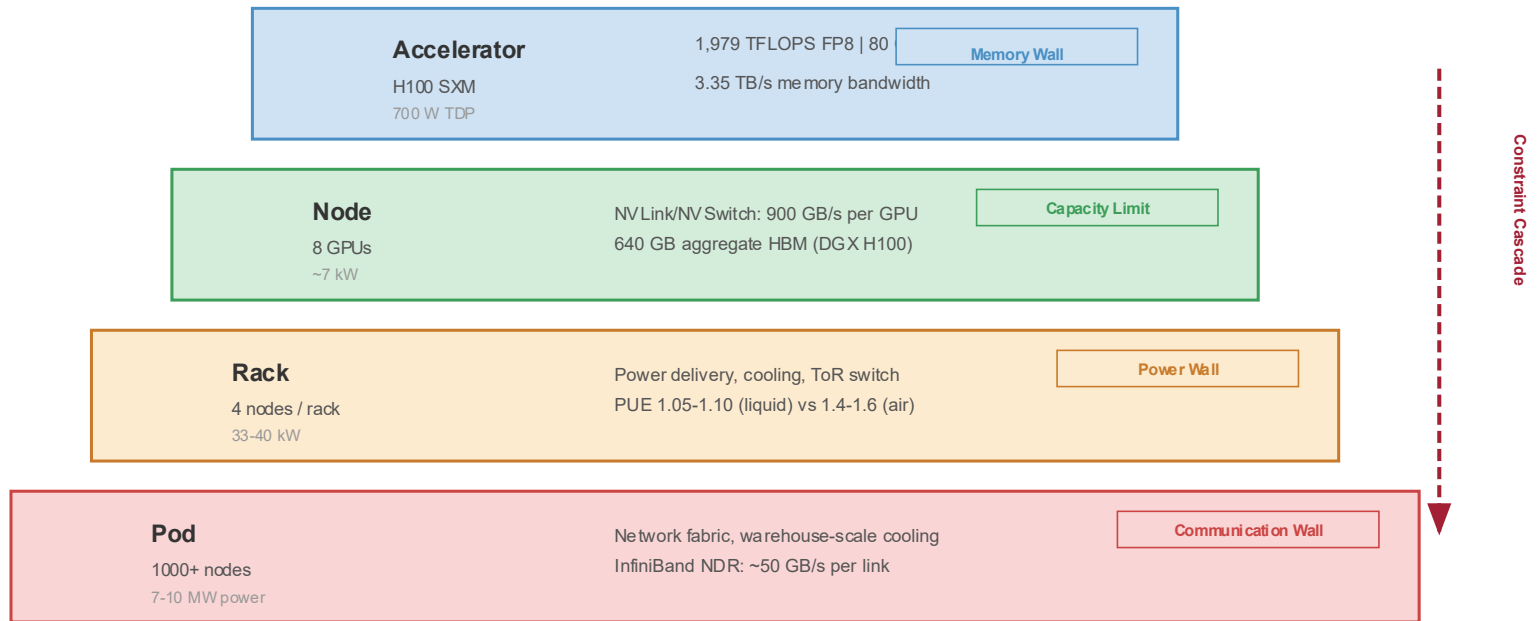
Material taken/derived from:

- Harvard ML Systems by Vijay Janapa Reddi
- @ 2026 released for use under a [CC BY-SA](#) license.

AI infrastructure stack

The Infrastructure Stack: From Transistor to Datacenter

Running example: 175B model → 350 GB weights (FP16)



Key principle: Scaling gives us more resources, but introduces a new bottleneck.

Three walls across the stack

Level	Memory Wall	Power Wall	Communication Wall
Accelerator	HBM bandwidth	700 W TDP	On-die mesh
Node	640 GB aggregate	7 kW to 10 kW	NVLink: 900 GB/s
Rack	PCIe to host DRAM	33–40 kW/rack	ToR switch
Pod	Distributed storage	7–10 MW	IB: 50 GB/s

Running example: 175B model → 350 GB weights (FP16) → 2.8 TB training state → does not fit in one chip

Node & bandwidth

The node: Aggregating accelerators

Why node: HBM has bandwidth but not enough capacity.

Component	175B Model
Weights (FP16)	350 GB
Gradients	350 GB
Optimizer (Adam)	2.1 TB
Total	2.8 TB
Single H100 HBM	80 GB
8-GPU Node HBM	640 GB

Still not enough for full training state → ZeRO sharding, offloading

DGX H100 node:

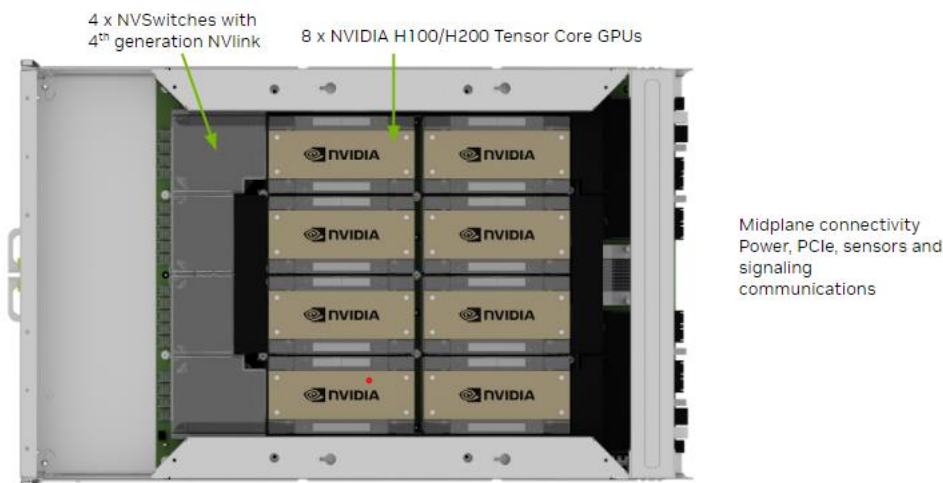
8× H100 SXM GPUs

640 GB aggregate HBM

NVSwitch: 900 GB/s per GPU

4× NVSwitch chips

Total: ~10.2 kW



<https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html>

The node: Aggregating accelerators

Why node: HBM has bandwidth but not enough capacity.

Component	175B Model
Weights (FP16)	350 GB
Gradients	350 GB
Optimizer (Adam)	2.1 TB
Total	2.8 TB
Single H100 HBM	80 GB
8-GPU Node HBM	640 GB

Still not enough for full training state →
ZeRO sharding, offloading

DGX H100 node:

8× H100 SXM GPUs

640 GB aggregate HBM

NVSwitch: 900 GB/s per GPU

4× NVSwitch chips

Total: ~10.2 kW

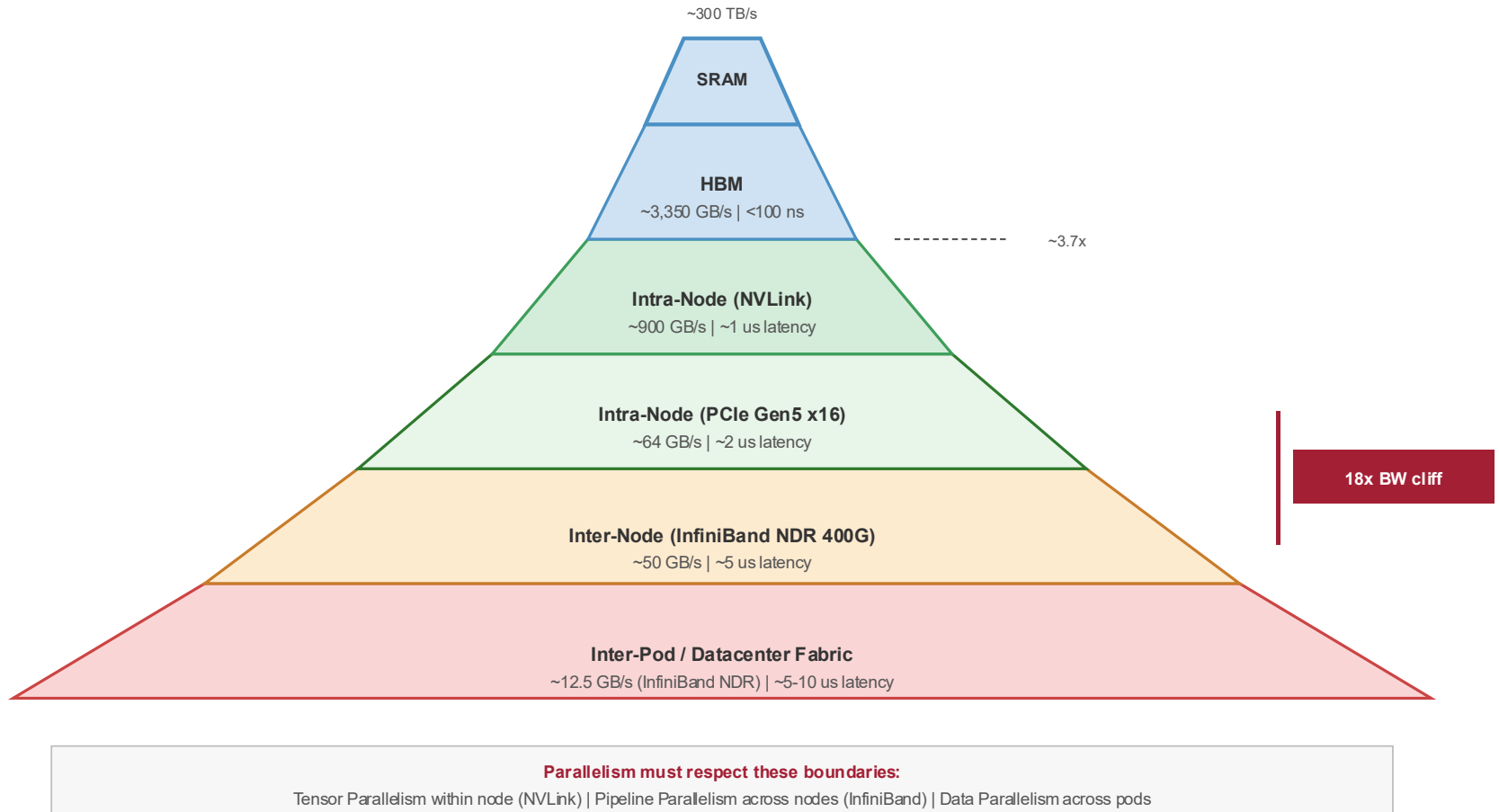
Key insight:

NVSwitch transforms 8 independent GPUs into one “**virtual accelerator**” with unified memory.

<https://docs.nvidia.com/dgx/dgxm100-user-guide/introduction-to-dgxm100.html>

The bandwidth hierarchy

The Bandwidth Hierarchy: Orders-of-Magnitude Staircase



The bandwidth hierarchy: The numbers

Domain	Interconnect	BW	Latency	Scope
Intra-Package	Si Interposer	~3,350 GB/s	<100 ns	Single chip
Intra-Node	NVLink 4.0	~900 GB/s	~1 μ s	8 GPUs
Intra-Node (IO)	PCIe Gen5 x16	~64 GB/s	~2 μ s	CPU–GPU
Inter-Node	IB NDR 400G	~50 GB/s	5–10 μ s	1000s GPUs

The 18x cliff: NVLink (900 GB/s) → InfiniBand (50 GB/s). This single boundary dictates the entire parallelism hierarchy for distributed training.

Rack

The rack: Power and cooling

AI rack power density: 10x traditional datacenter!

Component	Power
32 GPUs ($\times 700$ W)	22.4 kW
CPUs, NVSwitch, NICs	~ 8 kW
Power conversion losses	~ 3 kW
Total per rack	33–40 kW
Traditional server rack	5–10 kW

128 racks for 175B model training $\rightarrow$ 5 MW cluster

PUE: Cooling tax

$$\text{PUE} = \frac{\text{Total facility power}}{\text{IT equipment power}}$$

Air-cooled: PUE 1.4—1.6

(40—60% extra power overhead!)

Liquid-cooled: PUE 1.05—1.10

(5—10% overhead only)

For 1,000 GPUs: liquid cooling saves **294 kW** $\rightarrow$ \$258K/yr in electricity

Pod

The pod: Warehouse-scale computer

1,024-node DGX H100 cluster

Resource	Scale
GPUs	8,192
Racks	~250
Power draw	7–10 MW
Weight	300+ metric tons
IB switches	Hundreds
Cables	Tens of thousands

The datacenter floor must support 1,500—2,500 kg/m² (vs. 500—1,000 traditional)

175B model training time

300B tokens

Physics limit: ~**95.8 hour (4 days)**

(1,024 GPUs, FP8 peak, **45% MFU**)

Actual: **1—2 weeks!**

(**comm, bubbles, checkpoints, failures**)

Closing the gap is **distributed systems engineering**

Scaling efficiency

$$\eta_{scale} = \frac{T_{compute}}{T_{compute} + T_{comm} + T_{idle}}$$

GPUs	Ideal Speedup	Actual	η_{scale}
8	8×	7.6×	95%
64	64×	54×	85%
1,024	1,024×	615×	60%
8,192	8,192×	3,277×	40%

Amdahl at scale: The first 100 GPUs provide near-linear speedup. Beyond 2,000—4,000, marginal benefit per GPU approaches zero.

Your turn: Node memory budget

Memory budget: 70B model

Calculate per-node memory:

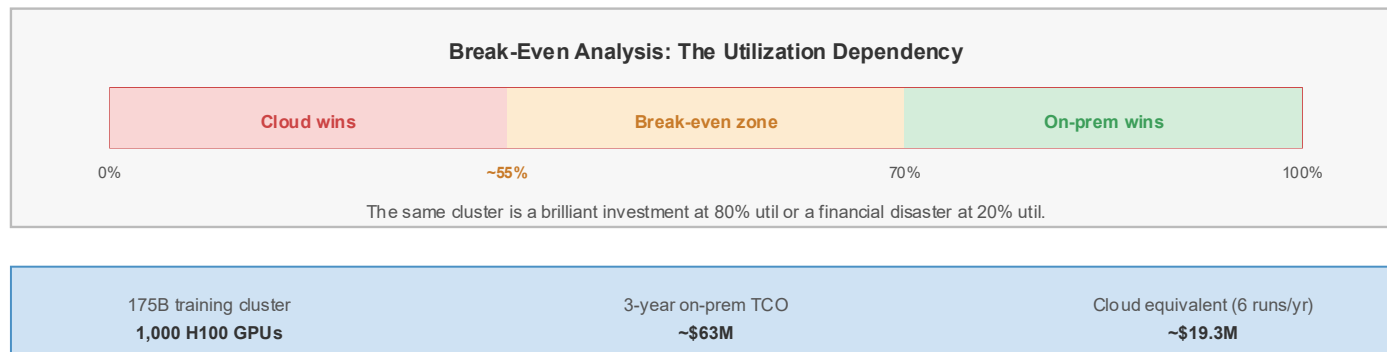
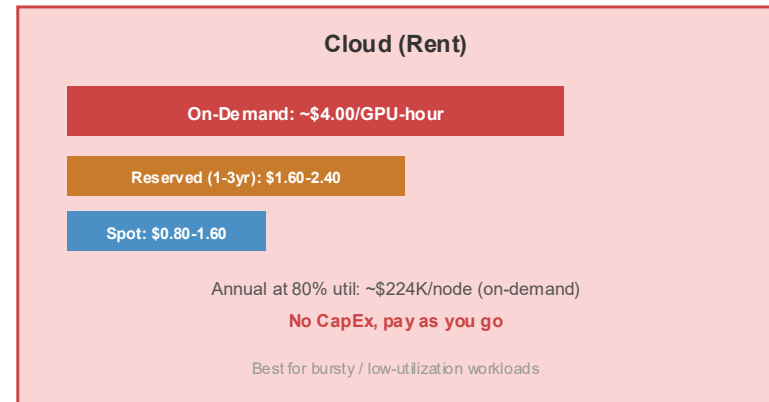
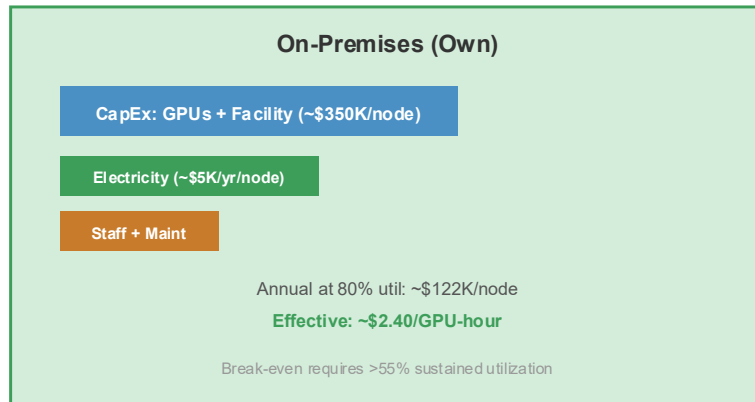
- Parameters: 70B x 2 bytes (FP16)
- Gradients: same as parameters
- Adam optimizer: $m + v + \text{FP32 copy} = 12$ bytes/param
- Activations: batch-dependent

Does it fit on one 8-GPU node (640 GB HBM)?

TCO (total cost of ownership)

Total cost of ownership

TCO: Build vs. Buy — The Utilization Question



Infrastructure economics: utilization is the single variable that flips the build-vs-buy decision.

Central tension: Utilization is the single variable that flips the build-vs-buy decision.

TCO: The numbers

	On-Premises	Cloud (On-Demand)
CapEx	\$350K/node (amortize 3 yr)	\$0
Electricity	~\$5K/yr/node	Included
Annual cost	~\$122K/node (80% util)	~\$224K/node (80% util)
Eff. rate	~\$2.18/GPU-hour	\$4.00/GPU-hour
Break-even	~43.5% sustained utilization	

Own if: continuous 24/7 workloads, predictable demand, 3+ year horizon

Rent if: bursty demand, rapid experimentation, <2 year horizon

Question: Build vs. buy

Build vs. buy decision

Your team plans to train a 70B model:

- 500 H100 GPUs for 3 weeks per run
- 4 training runs per year
- Cloud rate: \$4.00/GPU-hour
- On-prem: \$350K/node, 3-yr amortization

Calculate annual cost for both options

How many runs/year to justify on-prem?

Scaling laws

Scaling laws: What to predict

We have a fixed training compute budget

- How large should the model be?
- How many tokens should it train on?
- What changes if we get 4x more compute?

Goal: The lowest loss for the compute we spend

One budget, many choices

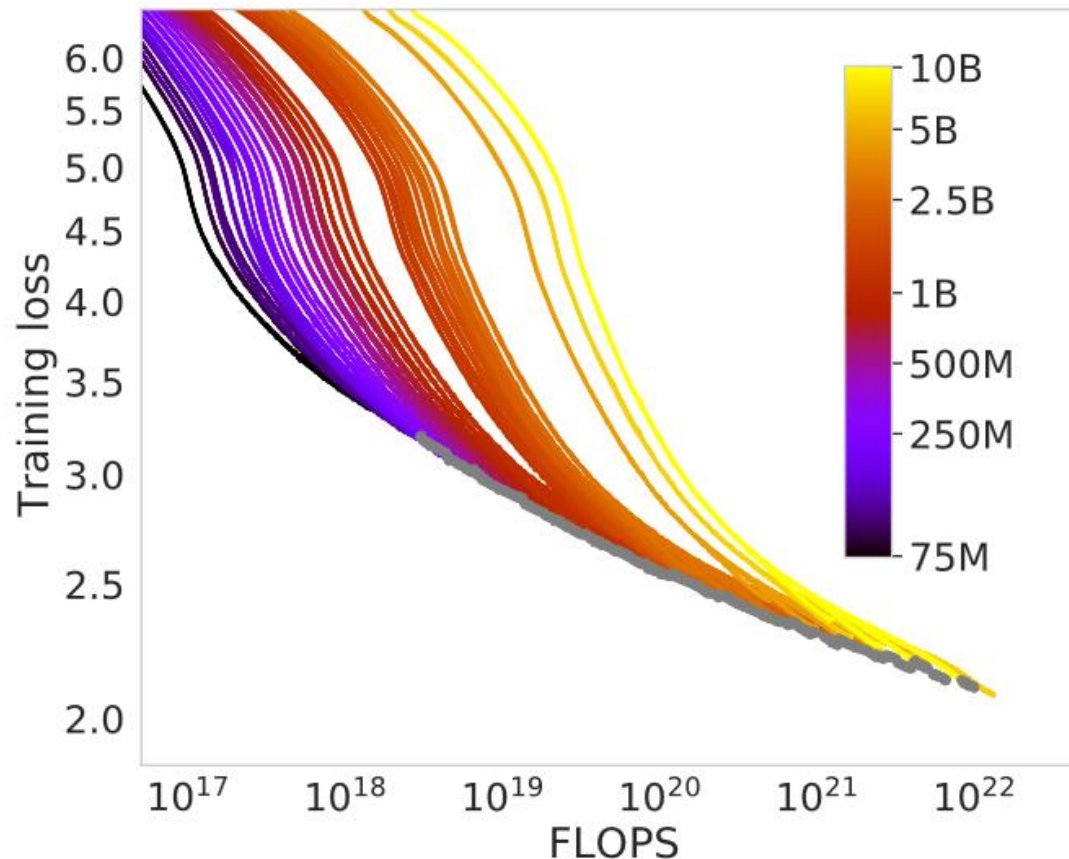
N = parameters D = training tokens

$$C \approx 6ND = 6 \times 10^{20} \text{ FLOPs}$$

Model size N	Tokens D
1B	100B
2B	50B
5B	20B

Which choice gives the lowest loss?

Approach 1: Minimum over training curves



Choose a model size N

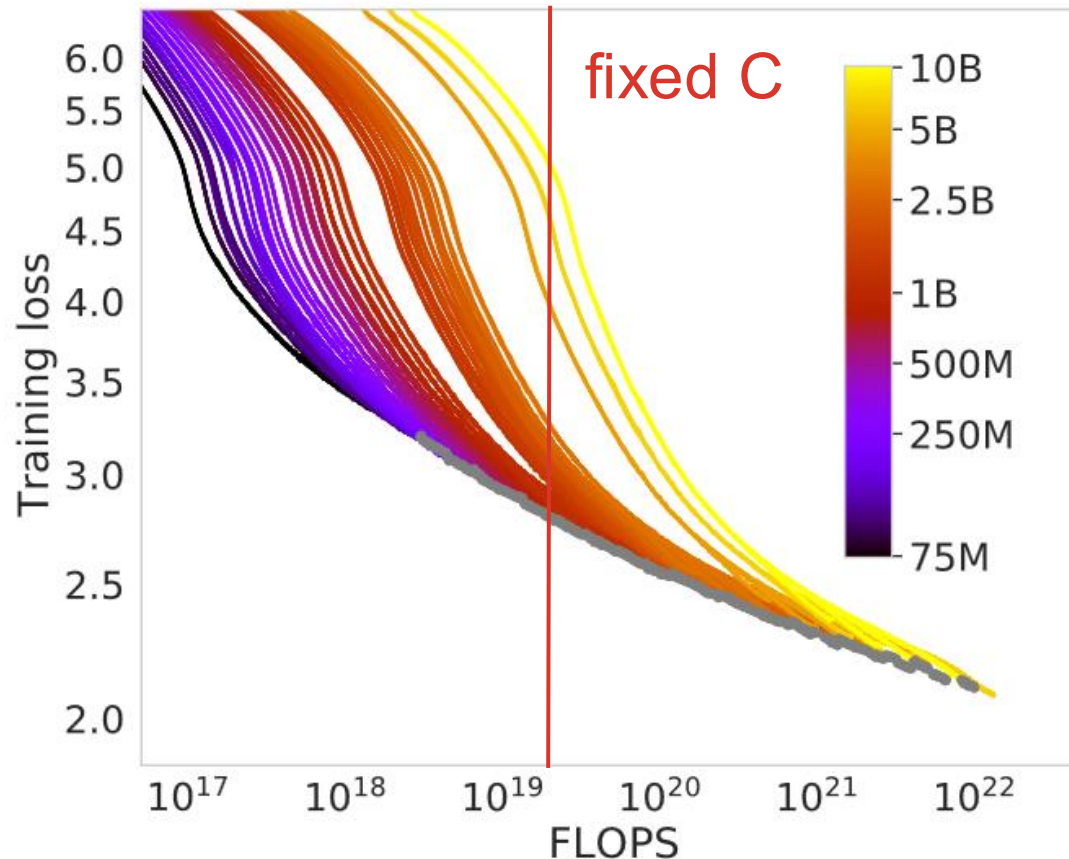
Train it with four training horizons

Repeat for other model sizes

Which run has the lowest loss at this budget?

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Approach 1: Minimum over training curves



Keep the lowest-loss point at each budget

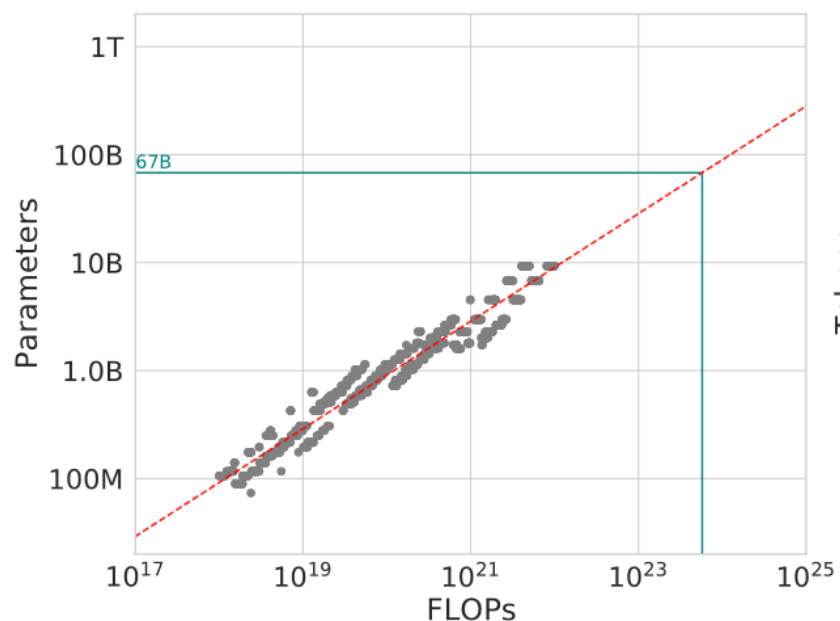
Record N and D at that point

Repeat across budgets to trace the lower envelope.

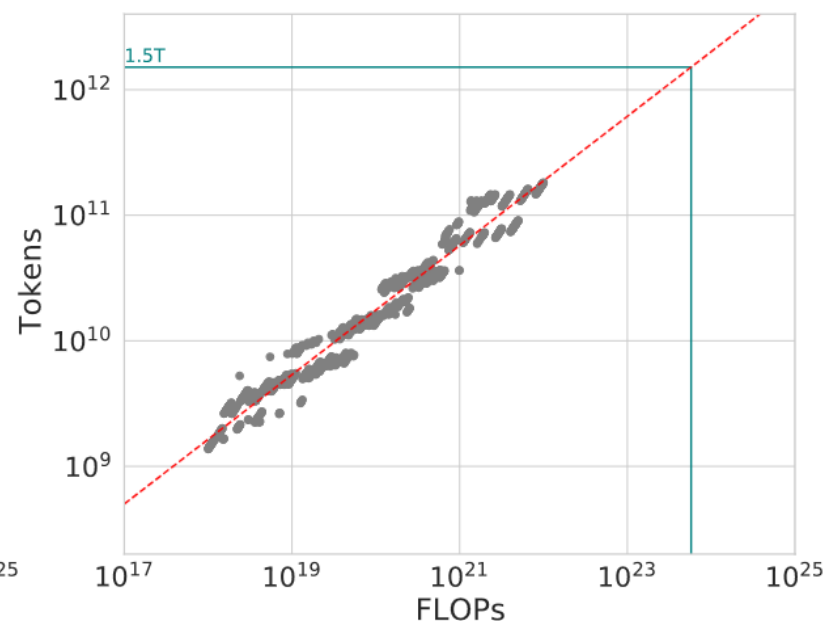
Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

How does the best allocation scale?

Optimal model size



Optimal training tokens



Small experiments guide a larger training run.

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Training stats of representative LLMs

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
<i>Chinchilla</i>	70 Billion	1.4 Trillion

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Approach 2: Equal compute (IsoFLOP)

N = parameters D = training tokens

$$C \approx 6ND = 6 \times 10^{20} \text{ FLOPs}$$

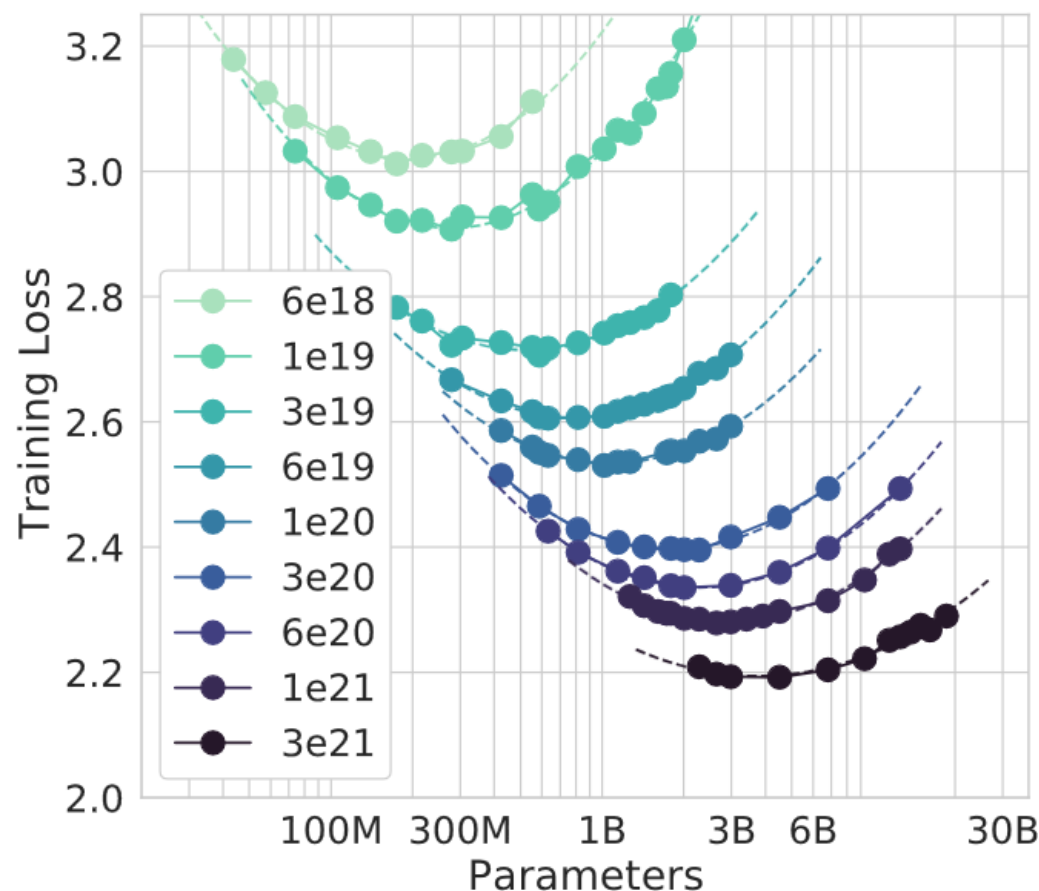
Model size N	Tokens D	Minimum loss
1B	100B	?
2B	50B	?
5B	20B	?

Each run finishes its learning-rate decay at its endpoint.

“IsoFLOP” means the same total training compute.

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Approach 2: A valley at each budget



One dot:
one completed run

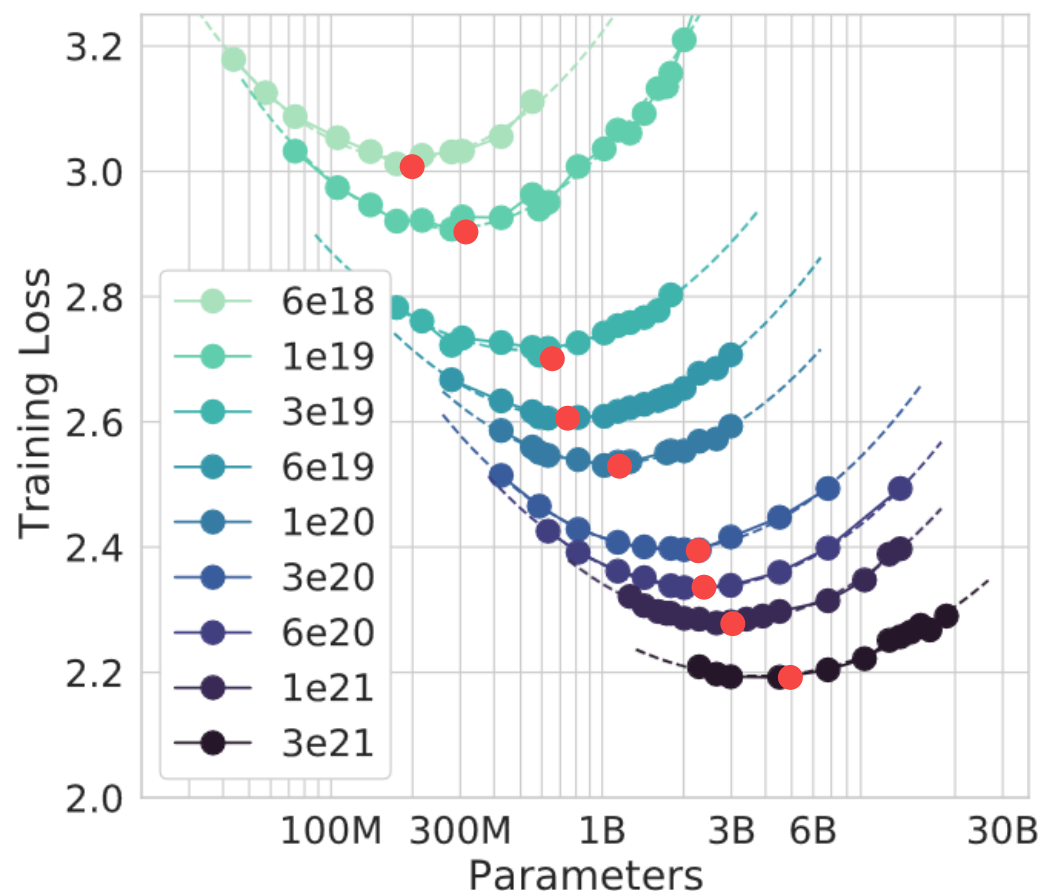
One color:
one compute
budget

The valley:
best model size

A larger model gets fewer tokens at fixed compute.

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Approach 2: A valley at each budget



One dot:
one completed run

One color:
one compute
budget

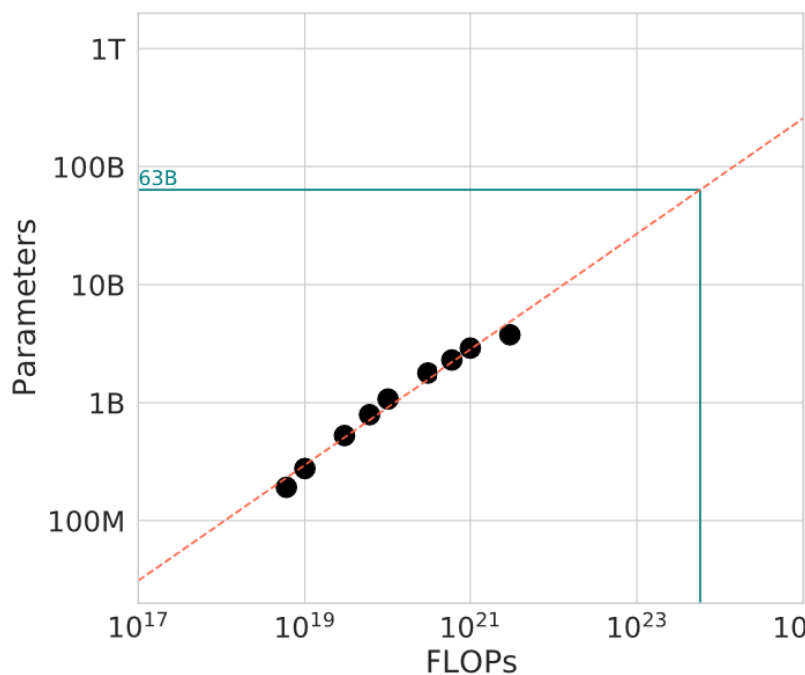
The valley:
best model size

A larger model gets fewer tokens at fixed compute.

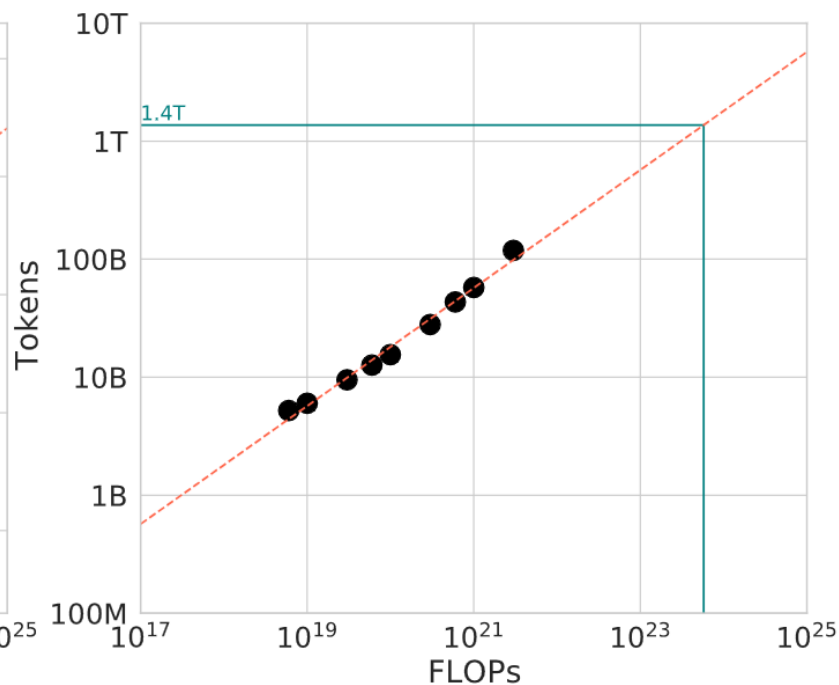
Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

How does the best allocation scale?

Optimal model size



Optimal training tokens



Small experiments guide a larger training run.

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Optimal model scaling

$$C \approx 6ND \longrightarrow N_{opt} \propto C^a \longrightarrow D_{opt} \propto C^b$$

Method	a	b
Training curves	0.50	0.50
IsoFLOP profiles	0.49	0.51
Loss-function fit	0.46	0.54

Approximately: $N_{opt} \propto \sqrt{C}$ and $D_{opt} \propto \sqrt{C}$



$$D_{opt} \propto N_{opt}$$

4× compute: about 2× parameters and 2× tokens

Table 3 | Estimated optimal training FLOPs and training tokens for various model sizes. For various model sizes, we show the projections from Approach 1 of how many FLOPs and training tokens would be needed to train compute-optimal models. The estimates for Approach 2 & 3 are similar (shown in [Section D.3](#))

Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Discussion: How much more data?

Baseline: 7B parameters, 140B training tokens

1. For a 70B model, how many tokens and how much training compute?
2. If compute doubles but tokens stay at 140B, how large can the model be? Is that the best allocation?

Discussion: Allocation matters

70B parameters needs about 1.4T tokens.

$10\times \text{parameters} \times 10\times \text{tokens} = 100\times \text{compute}$

With 2× compute	Parameters	Tokens
Keep tokens fixed	14B	140B
Scale both equally	≈9.9B	≈198B

More compute also changes the best data budget.

Illustrative calculations under $C \approx 6ND$ and equal scaling

A prediction for Gopher's budget

Gopher: 280B parameters, 300B tokens

Training budget: 5.76×10^{23} FLOPs

Predicted best model size:

40–70B parameters

Can a smaller model outperform Gopher?

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>

Approach 3

A fitted formula for loss

$$L(N, D) \approx E + A / N^\alpha + B / D^\beta$$

More parameters reduce the model-size term.

More training tokens reduce the data term.

Minimize predicted loss at fixed compute.

Hoffman et al. 2022. <https://arxiv.org/pdf/2203.15556>